— PORTFOLIO —

Portfolio.

BACKEND & INFRA ENGINEER

저점을 높이는 개발자

name.	김민욱
email.	kmwpcpk@gmail.com
tel.	010-3389-6730
github.	github.com/kr-Uk

김민욱

백엔드 개발자



name. 김민욱

email. kmwpcpk@gmail.com

tel. 010-3389-6730

github. github.com/kr-Uk

about me

최악의 한 순간을 줄이는 백엔드 개발자

높은 천장보다, 무너지지 않는 바닥을 먼저 생각하는 백엔드 개발자입니다.
과금이 급증하던 서비스의 백엔드를 Firebase에서 Spring으로 전면 재설계하고,
배포 중에도 서비스가 멈추지 않는 무중단 배포 환경과 장애를 즉시 감지하는 모니터링 및 백업 체계를 만들었습니다.
쿼리 로그, 과금 지표, 시스템 메트릭으로 병목을 찾아 장애와 비용이 터지는 순간을 미리 줄입니다.

education

2025.07 ~ 2026.06	**삼성청년 소프트웨어 AI 아카데미 14기 (SSAFY 모바일 트랙)**
2018.02 ~ 2025.02	**한동대학교 컴퓨터공학 전공**

project summary

DotShelf
멀티플랫폼 독서기록, 가상책장, 인프라 총괄 + 백엔드 리더

BoneToBe
AI 골격 분석 의류 추천, 백엔드 (커뮤니티 및 모니터링)

ITTO
위치 기반 소셜 매칭, 백엔드 1인 (Firebase→Spring 전환)

certifications

2025.06	**정보처리기사**
2024.11	**OPIc (IM2)**
2024.05	**우수논문상 (동상)**

skills

Java · **Spring Boot** · **Postgres SQL** · **MySQL** · **Docker**

DotShelf

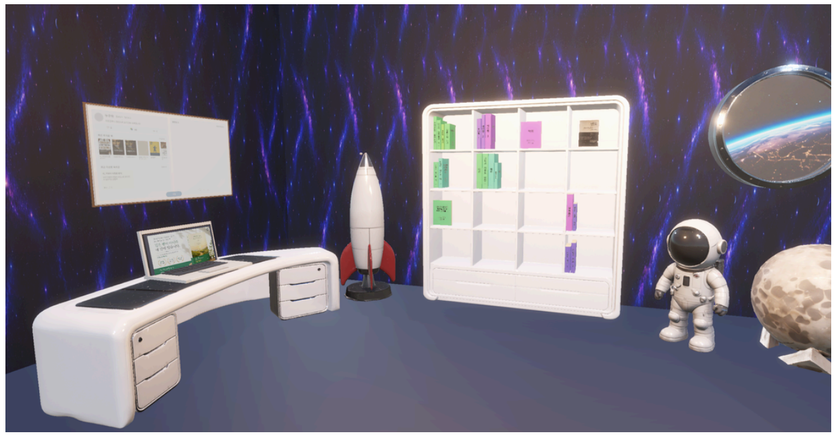
읽은 책을 기록하고 나만의 가상 책장을 꾸미며 공유하는, 웹·모바일·데스크톱 멀티 플랫폼 독서 서비스입니다.

project description

주제	멀티플랫폼 독서 기록 + 가상 책장 꾸미기
배경	웹·Unity·데스크톱이 공통 소비하는 REST API와 안정적 무중단 운영 인프라가 필요
기간	2026.04 ~ 2026.05 SSAFY 자율 프로젝트
인원	6인 (BE 2), 인프라 총괄 + 백엔드 리더
스택	Java 21 · Spring Boot 4 · PostgresSQL · Docker · GitLab CI/CD · Tailscale

기여한 부분

인프라 전반을 총괄했습니다. 2-tier 배포 구조, Blue-Green 무중단 배포, GitLab CI/CD 파이프라인, 모니터링 스택, 서버 보안 하드닝을 설계·운영하고 백엔드 리더로 팀 컨벤션과 REST API 일부를 담당했습니다.



/ 가상 책장 · 우주 테마



/ 가상 책장 · 스푸키 테마

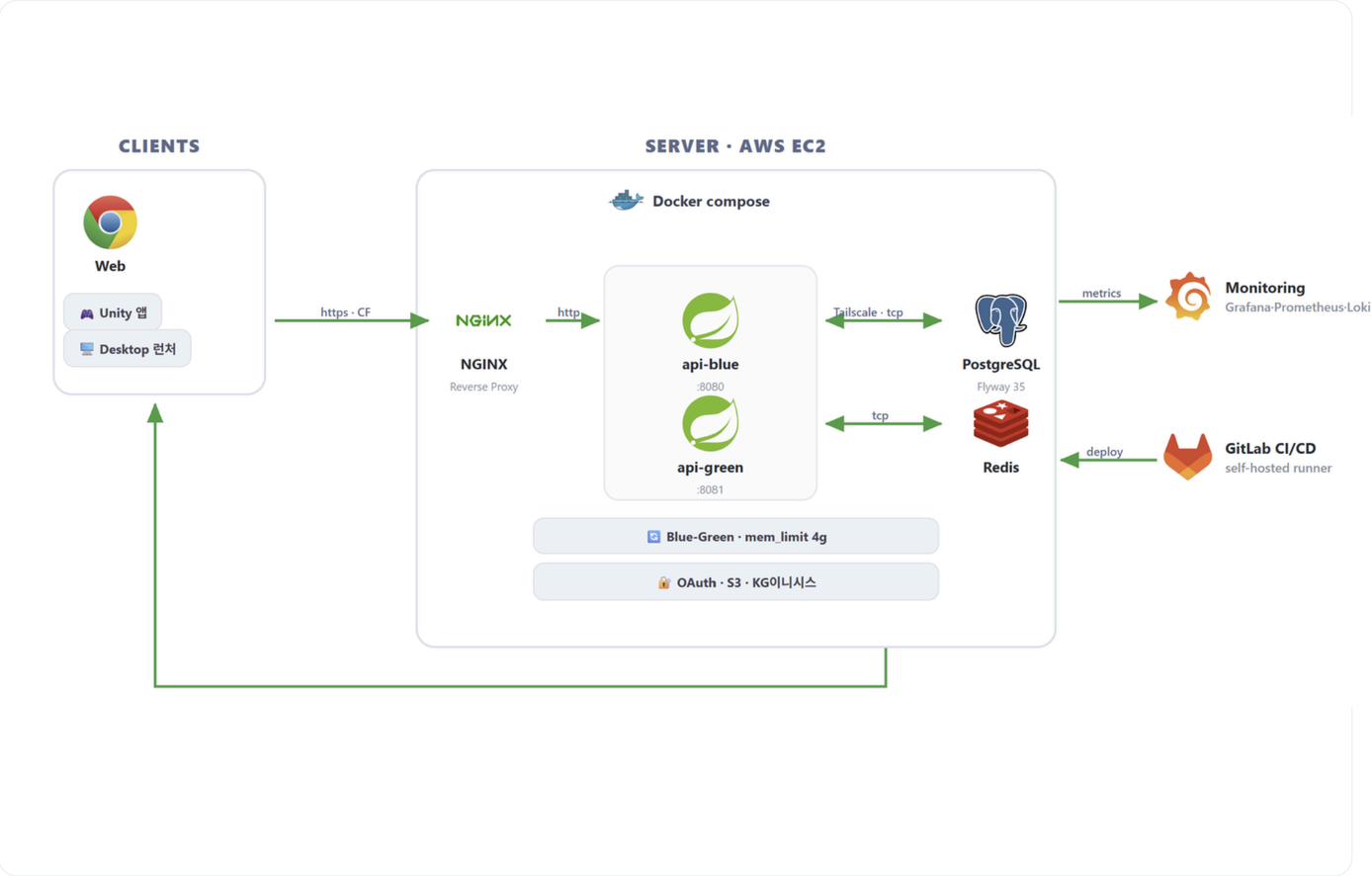
서비스 아키텍처

설계 의도

학기말 소멸하는 SSAFY 서버와 영속 서버를 두고 **"변하면 안 되는 것은 오래 사는 서버에"** 원칙으로 App·DB를 2-tier 분리했습니다. 두 서버 간 DB/Redis는 공용 인터넷에 노출하지 않고 Tailscale 사설망에만 bind했습니다.

데이터 흐름

Web·Unity·Desktop → Cloudflare → Nginx(allowlist) → api-blue:8080 / api-green:8081 → PostgreSQL(Flyway 35), Redis. 메트릭/로그는 Prometheus·Grafana·Loki로 수집합니다.



Blue-Green 무중단 배포 - 배포 중 다운타임 제거

단일 컨테이너 배포는 재기동 동안 다운타임이 발생했습니다. api-blue(8080), api-green(8081) 두 색을 공존시키고, 비활성 색을 먼저 기동→헬스 통과 대기 →Nginx upstream 교체 후 reload로 1~5초 내 graceful cutover하도록 설계했습니다. deploy-blue-green.sh가 활성 색 감지부터 스위칭,smoke,구 색 stop까지 오케스트레이션합니다.

STEP · 1

비활성 색 기동 + 헬스체크

api-green :8081

poll-health.sh 대기

▶

STEP · 2

Nginx upstream 교체 · reload

upstream.active.conf 교체

nginx -s reload (1~5s)

▶

STEP · 3

구 색 stop 무중단 전환 완료

2분 유예 후 구 색 종료

downtime 0

트러블슈팅

Blue-Green 도입의 선행 조건 - JVM heap이 컨테이너가 아닌 호스트 기준으로 잡혀 OOM 위험을 사전 차단했습니다.

PROBLEM 문제

두 색 공존 시 OOM 발생 가능

Blue-Green으로 api-blue·api-green 두 컨테이너가 동시에 떠 있을 때, 각 JVM heap이 과도하게 커져 호스트 메모리(15GiB)를 초과할 위험이 있었습니다.

CAUSE 원인

mem_limit 부재로 heap이 호스트 기준

MaxRAMPercentage=60은 컨테이너 제한의 60%를 heap max로 잡는데, compose에 mem_limit 이 없어 컨테이너 제한이 호스트 전체로 취급 → heap이 ~9GiB까지 가능. 2×9 = 18GiB > 15GiB.

FIX 해결

mem_limit: 4g 명시

컨테이너 메모리 제한을 4g로 명시해 heap을 컨테이너 기준으로 고정했습니다. 이를 Blue-Green 도입의 선행 조건으로 처리했습니다.

RESULT 결과

2색 공존에도 OOM 0건

두 색 합산 메모리가 호스트 한도 내로 제한되어 무중단 배포 중에도 OOM 없이 안정 운영. Grafana에서 color별 JVM heap을 분리 관측해 검증했습니다.

Bone：Tobe

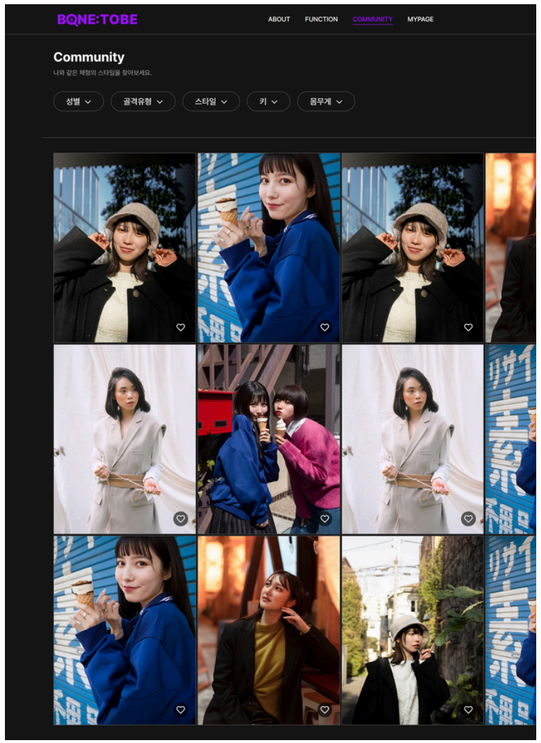
AI 체형 분석으로 골격 유형(스트레이트/웨이브/내추럴)을 판별하고 맞춤 의류를 추천하는 서비스입니다.

project description

주제	AI 골격 분석 기반 의류 추천
배경	체형 판별 + 커뮤니티로 스타일 정보를 공유하는 패션 서비스
기간	2026.01 ~ 2026.02 6주
인원	6인 (BE 3) 커뮤니티 API + 모니터링 담당
스택	Java 17, Spring Boot 3.5, PostgresSQL, Docker, Grafana, Loki, aws S3

기여한 부분

커뮤니티 API 설계 및 구현(피드 CRUD + 필터링), S3 Presigned URL 이미지 업로드, 그리고 팀 전원이 쓰는 Grafana + Loki **모니터링 환경**을 구축했습니다.



/ 커뮤니티 피드



/ 체형 분석 리포트

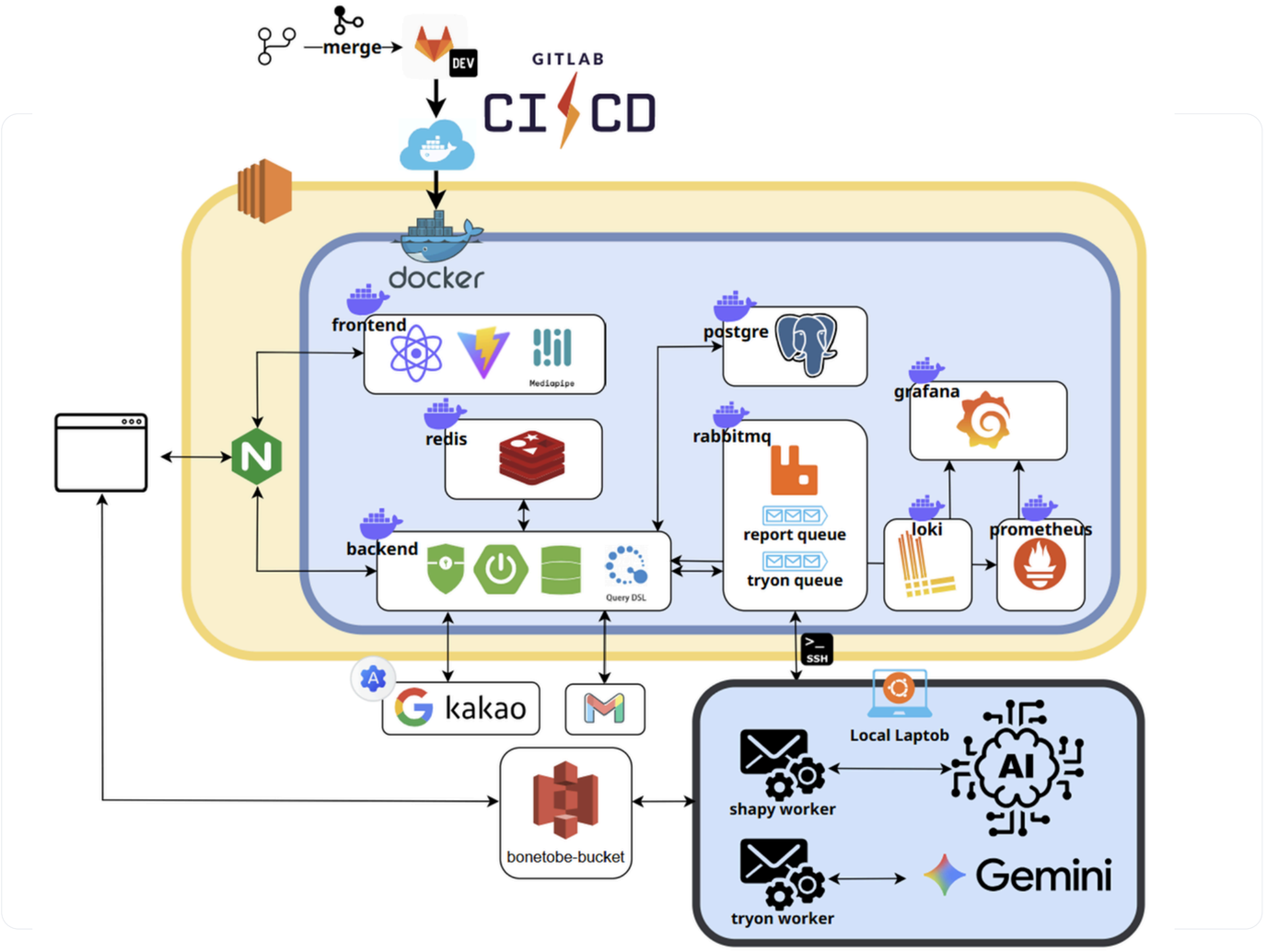
서비스 아키텍처

설계 의도

AI 체형 분석은 응답이 느려, 요청을 **RabbitMQ로 비동기** 처리해 API 스레드를 막지 않도록 분리했습니다. 팀 협업 병목은 공유 모니터링으로 구조적으로 해결했습니다.

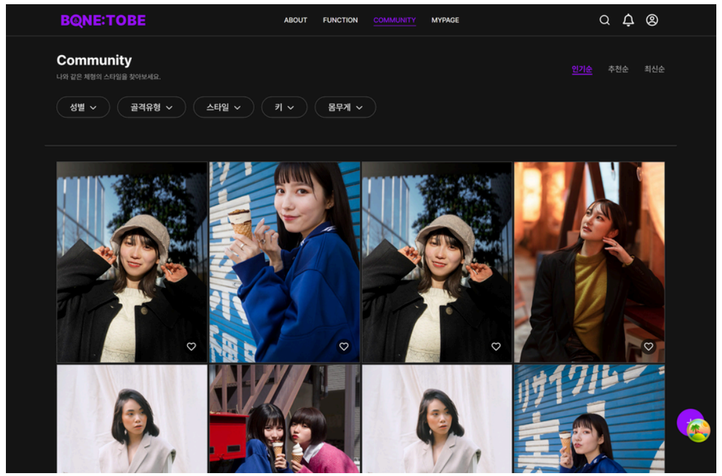
데이터 흐름

App → Spring Boot(REST) → PostgreSQL, S3(Presigned) / AI 요청은 RabbitMQ → AI 서비스. 메트릭 및 로그는 Prometheus, Grafana, Loki, Promtail로 중앙 집계합니다.

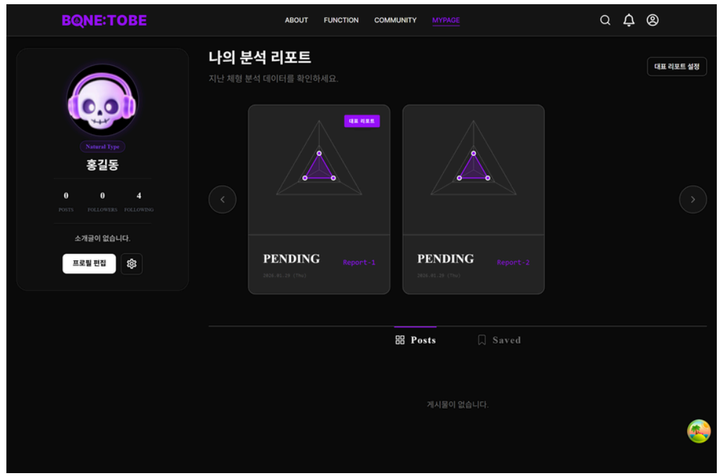


커뮤니티 피드 - 필터, 태그, 이미지 한 번에

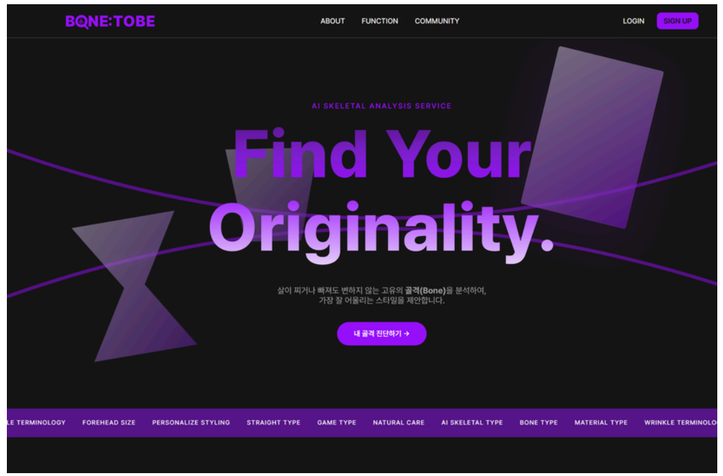
게시글 + 스타일 태그 + 좋아요 여부 + S3 이미지 URL을 함께 반환하는 피드를 설계했습니다. QueryDSL Multi-Stage 쿼리(ID 선조회 → 태그 집계 → Presigned URL 배치 생성)와 likeCount 비정규화로 조회 성능과 동시성 안전성을 함께 확보했습니다.



/ 커뮤니티 피드 · 필터



/ 프로필 · 분석 리포트



/ 메인 랜딩

트러블슈팅

피드 조회의 N+1 문제를 QueryDSL Multi-Stage 쿼리로 풀어, 쿼리 수를 95% 줄였습니다.

PROBLEM 문제

20개 게시글에 60개 이상 쿼리

피드 조회 시 게시글 + 태그 + 좋아요 + 이미지 URL을 함께 반환해야 했는데, Hibernate SQL 로그상 20개 조회에 61 개 이상의 쿼리가 실행됐습니다.

CAUSE 원인

Lazy Loading N+1 + 동적 필터

Post가 다대다/일대다/일대일 연관을 가져 지연 로딩마다 추가 쿼리(1 + 20×3)가 발생했고, 동적 필터가 결합돼 단순 Fetch Join으로는 해결이 어려웠습니다.

FIX 해결

QueryDSL Multi-Stage 전략

① 필터로 ID만 경량 조회 → ② ID 기반 JOIN + GroupBy.list()로 태그 집계 → ③ Facade에서 파일 ID 배치 수집 후 S3 URL 일괄 생성.

RESULT 결과

쿼리 61개 → 3개 · 응답 60%↓

쿼리가 약 95% 감소하고 S3 URL 생성도 20회 → 1회로 축소. Prometheus 메트릭 기준 피드 API 응답시간이 약 60% 단축됐습니다.

Itto

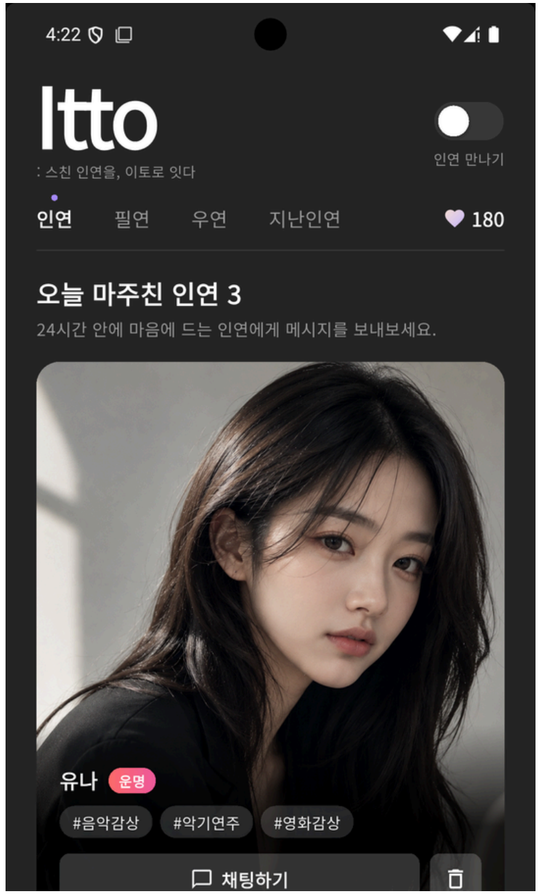
GPS로 주변 사용자와의 스침(만남)을 감지하고, 관계를 발전시켜 나가는 위치 기반 소셜 매칭 서비스입니다.

project description

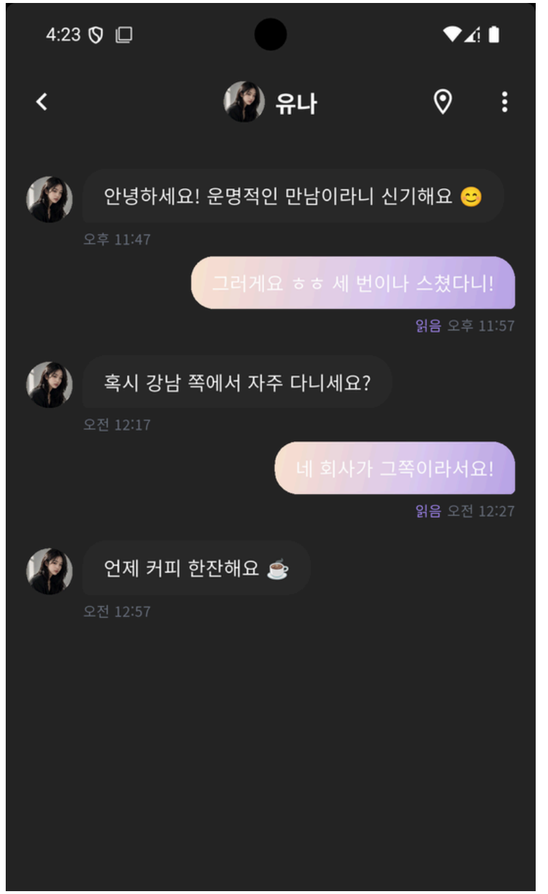
주제	위치 기반 소셜 매칭
배경	Firebase MVP의 과금 한계를 직접 분석 → Spring Boot로 전면 재설계
기간	2026.02 ~ 진행중
인원	1인 (풀스택) 백엔드, 앱, 인프라 전체
스택	Java 21 · Spring Boot 3.5 · PostgresSQL · Docker · FCM · Uber H3

기여한 부분

백엔드 1인 개발로 **H3 공간 인덱싱 위치 매칭**, JWT + Redis 토큰 즉시 무효화 체계, 결제/구독 시스템을 설계 구현하고 인프라까지 단독으로 운영했습니다.



/ 오늘 마주친 인연



/ 실시간 채팅

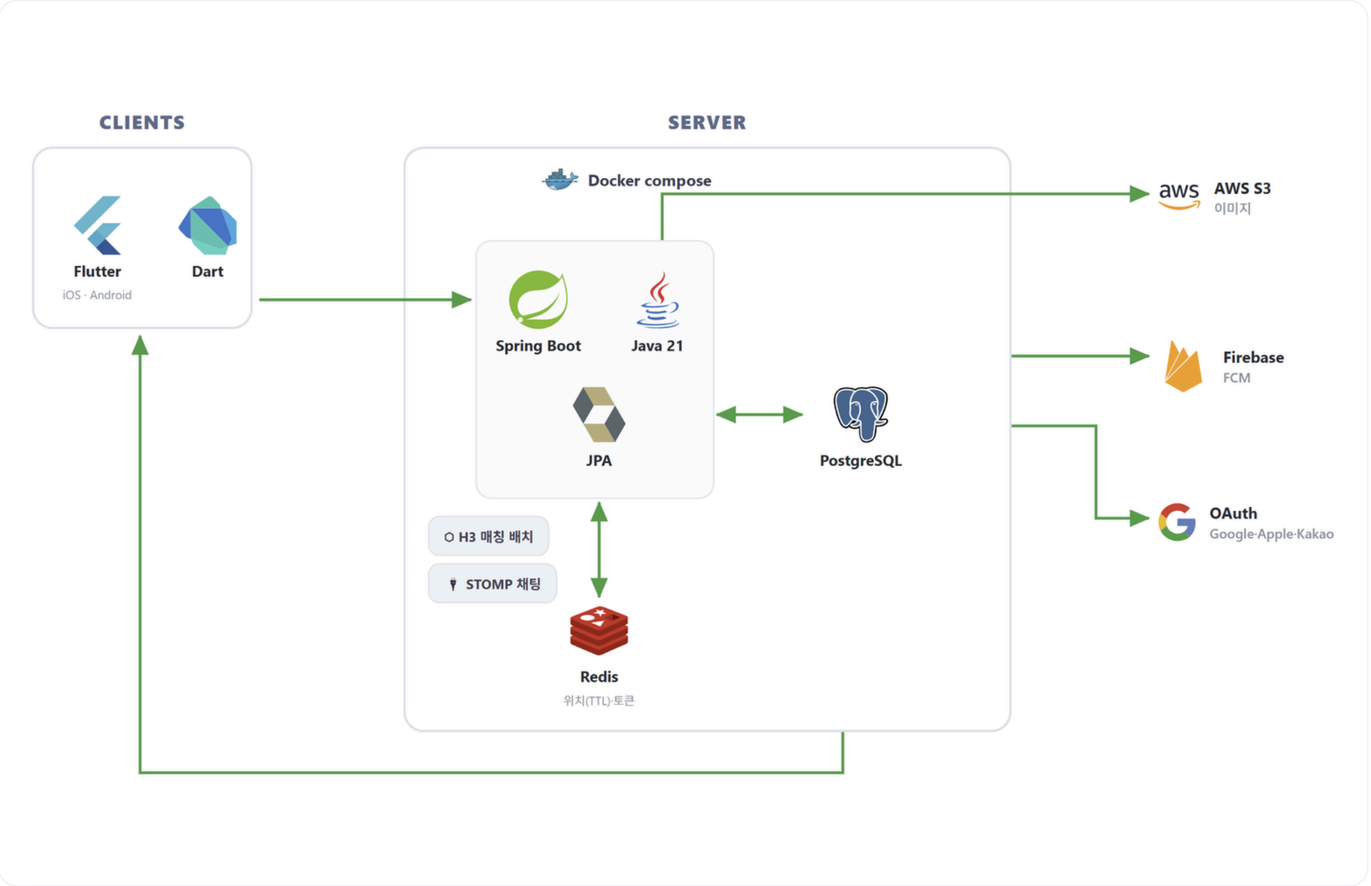
서비스 아키텍처

설계 의도

Firebase 분산 로직을 걷어내고 **헥사고날 + DDD** 14개 도메인으로 관심사를 분리했습니다. 위치 데이터는 DB 쓰기 대신 Redis(10분 TTL)로만 관리해 과금을 제거했습니다.

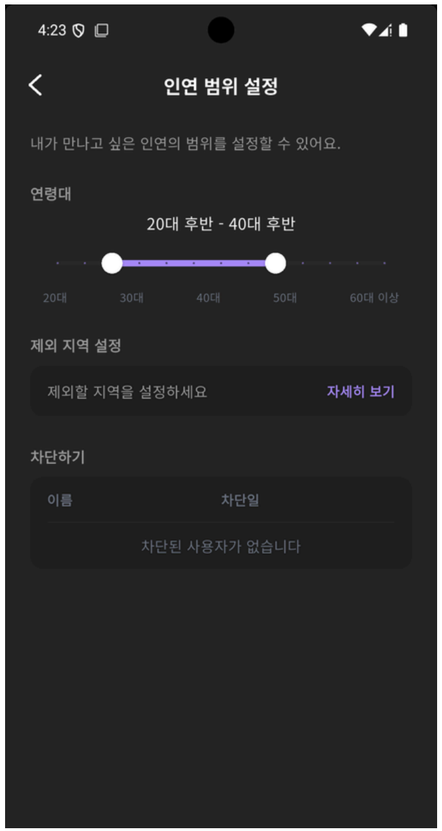
데이터 흐름

Flutter 앱 → Spring Boot → PostgreSQL(영속, Flyway), Redis(위치/H3, 토큰). 5분 배치 스케줄러가 H3 셀 단위로 스침을 감지하고 FCM으로 알립니다.

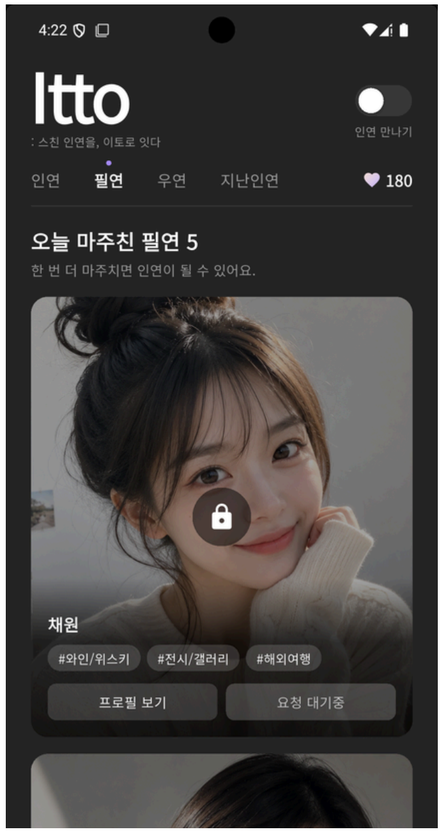


H3 공간 인덱싱 매칭 - O(n²) → O(n)

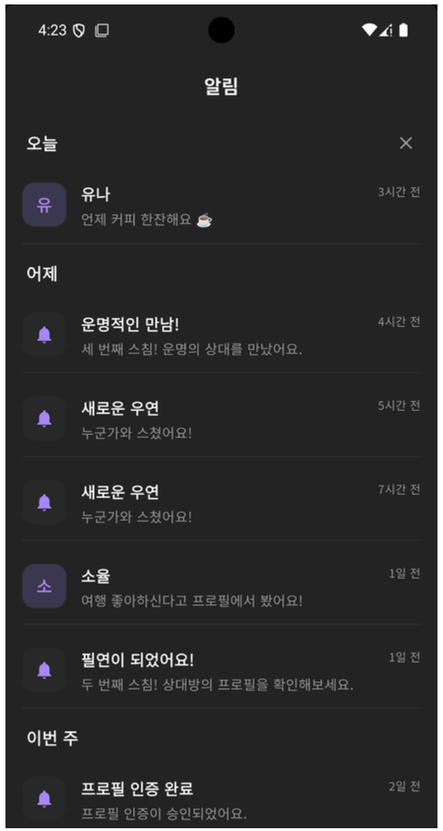
전수 비교(O(n²))로는 5분 배치 주기 내 매칭이 불가능했습니다. GPS를 Uber H3 셀(res 9, 약 174m)로 변환하고 Redis 이중 구조로 같은 셀 사용자끼리만 비교, Haversine으로 100m 정밀 검증 + 30분 쿨다운으로 중복을 제거했습니다.



/ 인연 범위 · H3



/ 스침 감지 결과



/ 스침 알림 · FCM

트러블슈팅

Firebase 과금 구조를 직접 분석하고, 서버리스의 한계를 Spring Boot 백엔드로 전면 전환해 해결했습니다.

PROBLEM 문제

Firestore 과금 폭증 + 로직 분산

실시간 위치의 잦은 읽기/쓰기로 일일 쓰기가 무료 한도(2만 건)를 초과했고, 로직이 클라이언트와 Cloud Functions 3개에 흩어져 유지보수가 어려웠습니다.

CAUSE 원인

건당 과금 + 서버리스 한계

Firestore는 읽기/쓰기 건당 과금이라 1분 간격 위치 업데이트만으로 일 15만 건 이상 누적. Realtime DB 전환으로 70% 절감했으나 서버 로직 부재는 그대로였습니다.

FIX 해결

Spring + PostgreSQL + Redis 재설계

헥사고날/DDD 14개 도메인으로 분리하고 위치는 Redis(10분 TTL)로 옮겨 DB 쓰기를 제거. 영속 데이터는 PostgreSQL + Flyway(9버전)로 관리했습니다.

RESULT 결과

위치 과금 0 · 변경 영향 최소화

위치 데이터 과금이 완전히 제거되고, 로직이 서버에 집중돼 신규 기능 추가 시 수정 파일이 평균 1~2개 모듈로 한정됐습니다. 마이그레이션 9회 무중단 적용.